

UE-INF2321M Animation, Corps Articulés Et Moteurs Physiques



Université Claude Bernard



Lyon 1

COTTIER Alexandre, P2018147

Vidéo démo : <https://drive.google.com/drive/folders/1cVRgldJz7exwME5FBPBB2q-q8ZztaGb4?usp=sharing>

1. Introduction

J'ai effectué l'implémentation d'une stratégie d'optimisation automatique des paramètres de la machine à états (FSM), l'amélioration de la qualité du mouvement par interpolation non-linéaire, l'ajout de fonctionnalités d'interaction (contrôle de vitesse) et de simulation physique (fluides/tissus), et aussi l'ajout d'une FSM (à genou puis se relève).

2. Stratégie d'Optimisation

Pour l'algorithme d'optimisation pour régler les 42 paramètres angulaires de la marche, j'ai fait dans la boucle principale un algorithme de type *Hill Climbing* avec mutation adaptative (je modifiais en dessous mais ce n'est pas vraiment très important d'en parler c'était surtout pour faire des tests différents pour bien voir les différences de changement).

Je pars d'une solution existante (les angles définis dans le code), j'applique une perturbation aléatoire (mutation) à chaque itération, j'évalue la performance sur une simulation courte, puis je ne conserve les nouveaux paramètres que si le coût total est strictement inférieur au meilleur coût connu.

Pour la mutation :

Pour éviter que le bipède ne "diverge" ou n'explose physiquement, j'ai fait une décroissance exponentielle de l'amplitude de mutation (dans certains tests).

- *La modification* -> : `float maxrand = exp(-m_optiData.nblter * 0.001);`

2.2. Cost function

La formule implémentée (pour la course) est :

$$\text{Coût}_{\text{Total}} = (P_{\text{dist}} * \text{-Distance}) + (P_{\text{nrj}} * \text{Energie}) + P_{\text{chute}}$$

Voici ma stratégie de pondération :

- Maximisation de la distance : j'applique un poids très élevé ($P_{\text{dist}} = 500$) pour que ce terme domine l'équation. Le signe négatif transforme le problème de maximisation en minimisation de coût.
- Contrôle de l'énergie : L'énergie brute étant très élevée ($\sim 2000+$), je l'ai normalisée avec un poids faible (P_{nrj} approx 1/50) pour qu'elle serve de critère secondaire (départager deux marches de même distance).
- Gestion de la chute (Penalty) : Plutôt que de rejeter systématiquement une simulation où le robot tombe, je fais une pénalité ($P_{\text{chute}} = 50$). L'algorithme accepte une chute *si et seulement si* le robot a parcouru une distance suffisante pour compenser cette pénalité (ça marche bien pour qu'il prenne du risque plus ou moins).

3. Amélioration de la Continuité : Interpolation Hermite

Le contrôleur de base utilisait une interpolation linéaire entre les poses clés, générant des mouvements robotiques et des discontinuités de vitesse angulaire.

J'ai modifié la méthode Update pour faire une interpolation cubique de type SmoothStep.

L'algorithme transforme le ratio temporel linéaire t (compris entre 0 et 1) selon la formule :

$$f(t) = t^2 * (3 - 2t)$$

En lissant les accélérations au début et à la fin de chaque transition d'état, je réduits les pics de couple demandés aux moteurs PD, rendant la marche plus fluide et visuellement naturelle.

4. Fonctionnalités Interactives et Environnementales

J'ai ajouté deux modules.

4.1. Contrôle Temporel Temps-Réel (Time Warping)

J'ai fait un contrôle direct de la vitesse de locomotion via le clavier.

- J'ajoute une variable *speedMultiplier* dans la classe FSM. Dans la boucle de mise à jour, le pas de temps *dt* est modulé par ce facteur :

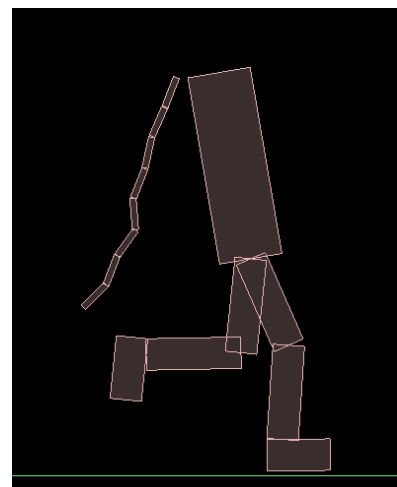
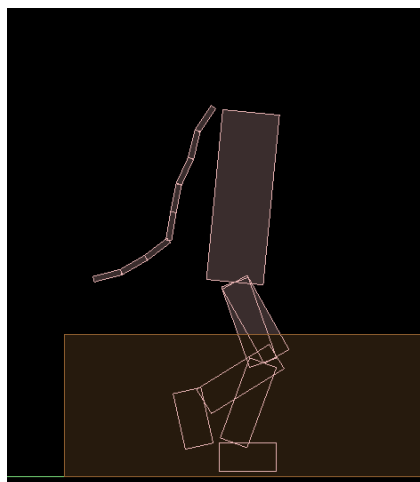
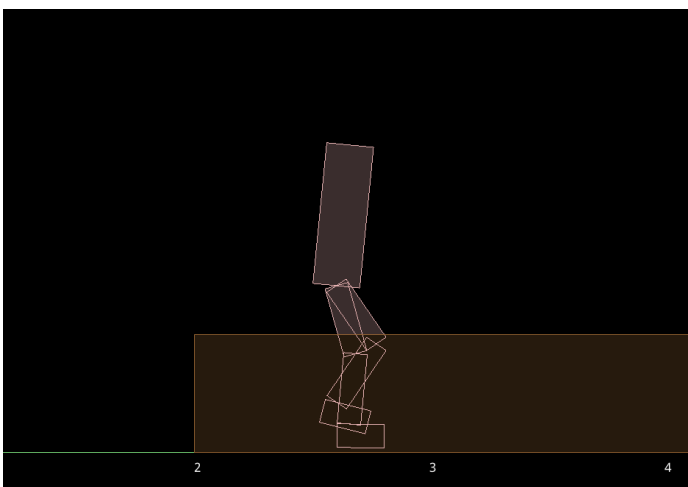
$$m_timeInState += Dt * speedMultiplier.$$

- Cela permet d'accélérer le personnage jusqu'à la course ou de le ralentir (effet "slow motion") sans changer les cibles angulaires, sachant qu'en état, quand j'augmente trop de speed il fait surtout du sur place, faudrait peut-être voir pour faire de l'optimisation avec ce paramètre de vitesse.

4.2. Interaction Fluide et Corps Déformables

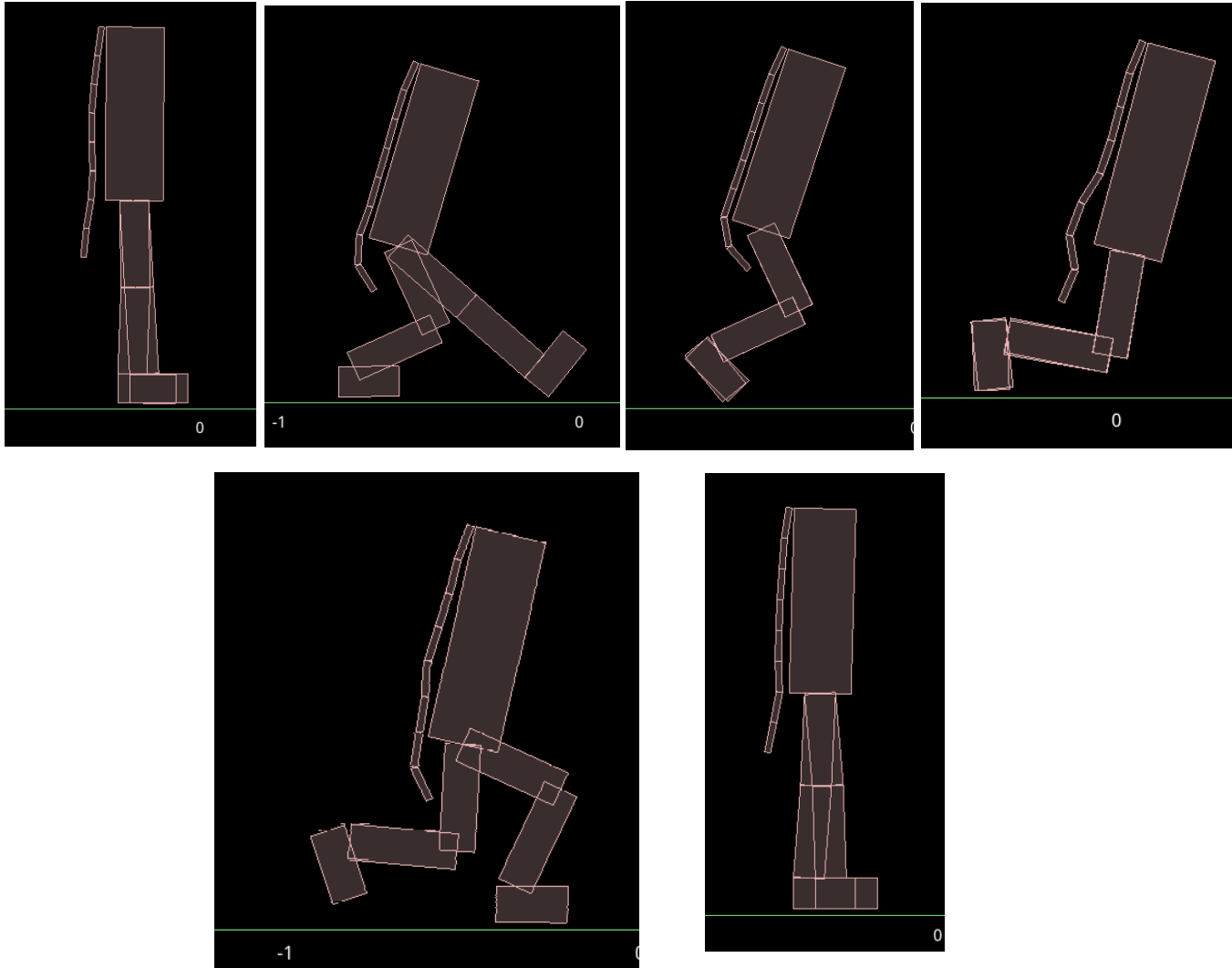
J'ai activé et paramétré des interactions physiques :

- Cape (Soft Body) : Ajout d'une chaîne de corps rigides liés par des articulations revolute sans moteur, simulant un tissu soumis à l'inertie et à la gravité.
- Boue (Fluid Drag) : Une force de traînée aérodynamique s'appliquant uniquement lorsque les membres entrent dans une zone définie ($y < 0$). La force ajoutée est proportionnelle au carré de la vitesse ($F = -k * v^2$), ce qui modifie la dynamique de marche et crée un effet personnage entraver dans de la boue (ou on peut voir ça comme un personnage qui marche dans une grosse étendue de neige aussi).



4.3. Simple extension avec une nouvelle machine à état (squat)

Il n'y a rien de compliqué à expliquer, c'est seulement un passage d'une jambe qui se plie puis la deuxième, passage sur les genoux, ensuite le personnage se relève en dépliant une jambe, puis fait revenir l'autre aussi, ce qui donne l'effet de s'agenouiller puis se relever tout en équilibre, fait aussi fait de l'optimisation sur cet état pour éviter le plus possible que le personnage tombe.



5. Résultats Obtenus

Grâce à la combinaison de l'optimisation des poids et de l'interpolation cubique, le personnage arrive à courir sur 11.80m en 5 secondes (sur 560 itérations).

Exemple de log quand je faisais l'optimisation pour la course.

A savoir qu'au bout d'un moment on n'arrive plus à obtenir de meilleur vraiment satisfaisant, si on laisse tourner l'algo c'est pour peaufiner au maximum mais souvent ça ne sert plus de continuer si par exemple on passe plus de 400 itérations sans amélioration (après on peut moduler avec la mutation etc mais voilà), par exemple entre l'itération 560 jusqu'à 920 je n'ai eu aucune amélioration, et celle que j'ai enfin eu à 920 ne m'améliore que de 0.01 mètre ce qui est rien du tout dans notre contexte.

```
>>> NOUVEAU RECORD ! Distance: 7.2881m
>>> NOUVEAU RECORD ! Distance: 8.3255m
>>> NOUVEAU RECORD ! Distance: 9.21073m
Iter: 10 | Dist: -2704.55 | Eng: 0 | Pen: 0 | TOT: -2704.55
>>> NOUVEAU RECORD ! Distance: 9.75532m
>>> NOUVEAU RECORD ! Distance: 10.2001m
Iter: 20 | Dist: -3528.3 | Eng: 0 | Pen: 0 | TOT: -3528.3
Iter: 30 | Dist: -2853.43 | Eng: 0 | Pen: 0 | TOT: -2853.43
Iter: 40 | Dist: -4909.81 | Eng: 0 | Pen: 0 | TOT: -4909.81
Iter: 50 | Dist: -3190.84 | Eng: 0 | Pen: 0 | TOT: -3190.84
Iter: 60 | Dist: -4436.34 | Eng: 0 | Pen: 0 | TOT: -4436.34
Iter: 70 | Dist: -4891.63 | Eng: 0 | Pen: 0 | TOT: -4891.63
>>> NOUVEAU RECORD ! Distance: 10.3271m
Iter: 80 | Dist: -3394.8 | Eng: 0 | Pen: 0 | TOT: -3394.8
Iter: 90 | Dist: -4442 | Eng: 0 | Pen: 0 | TOT: -4442
Iter: 100 | Dist: -3515.22 | Eng: 0 | Pen: 0 | TOT: -3515.22
Iter: 110 | Dist: -3105.1 | Eng: 0 | Pen: 0 | TOT: -3105.1
Iter: 120 | Dist: -4318.65 | Eng: 0 | Pen: 0 | TOT: -4318.65
Iter: 130 | Dist: -3632.05 | Eng: 0 | Pen: 0 | TOT: -3632.05
Iter: 140 | Dist: -4291.27 | Eng: 0 | Pen: 0 | TOT: -4291.27
Iter: 150 | Dist: -4962.9 | Eng: 0 | Pen: 0 | TOT: -4962.9
Iter: 160 | Dist: -5015.38 | Eng: 0 | Pen: 0 | TOT: -5015.38
>>> NOUVEAU RECORD ! Distance: 10.3807m
>>> NOUVEAU RECORD ! Distance: 10.7176m
Iter: 170 | Dist: -4853.17 | Eng: 0 | Pen: 0 | TOT: -4853.17
Iter: 180 | Dist: -4657.65 | Eng: 0 | Pen: 0 | TOT: -4657.65
>>> NOUVEAU RECORD ! Distance: 11.2362m
Iter: 190 | Dist: -3603.65 | Eng: 0 | Pen: 0 | TOT: -3603.65
Iter: 200 | Dist: -3930.5 | Eng: 0 | Pen: 0 | TOT: -3930.5
Iter: 210 | Dist: -4948.89 | Eng: 0 | Pen: 0 | TOT: -4948.89
Iter: 220 | Dist: -4474.14 | Eng: 0 | Pen: 0 | TOT: -4474.14
Iter: 230 | Dist: -4731.97 | Eng: 0 | Pen: 0 | TOT: -4731.97
Iter: 240 | Dist: -3664.97 | Eng: 0 | Pen: 0 | TOT: -3664.97
Iter: 250 | Dist: -4330.18 | Eng: 0 | Pen: 0 | TOT: -4330.18
Iter: 260 | Dist: -3987.24 | Eng: 0 | Pen: 0 | TOT: -3987.24
Iter: 270 | Dist: -4968.05 | Eng: 0 | Pen: 0 | TOT: -4968.05
>>> NOUVEAU RECORD ! Distance: 11.4752m
Iter: 280 | Dist: -3613.36 | Eng: 0 | Pen: 0 | TOT: -3613.36
Iter: 290 | Dist: -4742.25 | Eng: 0 | Pen: 0 | TOT: -4742.25
>>> NOUVEAU RECORD ! Distance: 11.6844m
Iter: 300 | Dist: -4263.09 | Eng: 0 | Pen: 0 | TOT: -4263.09
Iter: 310 | Dist: -4897.35 | Eng: 0 | Pen: 0 | TOT: -4897.35
Iter: 320 | Dist: -4727.53 | Eng: 0 | Pen: 0 | TOT: -4727.53
Iter: 330 | Dist: -4931.25 | Eng: 0 | Pen: 0 | TOT: -4931.25
Iter: 340 | Dist: -4372.09 | Eng: 0 | Pen: 0 | TOT: -4372.09
Iter: 350 | Dist: -5019.22 | Eng: 0 | Pen: 0 | TOT: -5019.22
Iter: 360 | Dist: -5308.28 | Eng: 0 | Pen: 0 | TOT: -5308.28
Iter: 370 | Dist: -3785.57 | Eng: 0 | Pen: 0 | TOT: -3785.57
Iter: 380 | Dist: -4637.44 | Eng: 0 | Pen: 0 | TOT: -4637.44
Iter: 390 | Dist: -4230.49 | Eng: 0 | Pen: 0 | TOT: -4230.49
Iter: 400 | Dist: -4939.42 | Eng: 0 | Pen: 0 | TOT: -4939.42
Iter: 410 | Dist: -4400.06 | Eng: 0 | Pen: 0 | TOT: -4400.06
Iter: 420 | Dist: -4987.24 | Eng: 0 | Pen: 0 | TOT: -4987.24
```